

Flask Day 3

Routing in Flask

PFS-HYD-054 | Flask Module

- 01 URL & Endpoints
- 02 Static & Dynamic Routes
- 03 URL Converters

```
app.py

@app.route('/students/<int:student_id>')
def profile(student_id):
    return f'Student: {student_id}'
```

Trainer

Madhu Parvathaneni, CTO - Codegnan
madhu@codegnan.com - 8121289993

Today's learning map

By the end of class, students should be able to design clean Flask URLs.



1. Understand routes

URL, endpoint, view function and request mapping.

2. Build static routes

Home, about, contact and support pages.

3. Build dynamic routes

Capture values from URLs using route variables.

4. Use converters

string, int, float, path and uuid converters.

5. Avoid route mistakes

Trailing slash, conflicts and wrong variable names.

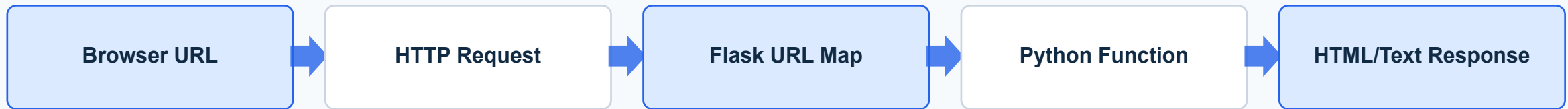
6. Practice labs

Create a mini student profile routing app.



Why routing matters

Routing decides what code runs for each URL typed by the user.



Real-world analogy

A college has many rooms. The address tells you which classroom to enter. In Flask, a URL tells the app which function to execute.

In Flask

A route is a rule that connects a URL path to a Python function. Without routes, Flask cannot decide what response to send.

Trainer focus

Do not let students memorize only syntax. Make them explain the complete path: URL -> decorator -> function -> response.

URL vs Endpoint vs View Function

Students commonly mix these terms. Separate them clearly.



URL Path

The visible address path requested by the browser.
Example: /about, /students/101

Endpoint

The internal name Flask uses for a route. By default, it is the function name.

View Function

The Python function that runs and returns the response.

```
1 @app.route('/about')
2 def about_page():
3     return 'About Codegnan'
```

Mapping

URL path: /about
Endpoint: about_page
View function: about_page()

Basic routing syntax

This is the base pattern students should write without hesitation.



```
1 from flask import Flask
2
3 app = Flask(__name__)
4
5 @app.route('/')
6 def home():
7     return 'Welcome to Flask Routing'
8
9 if __name__ == '__main__':
10     app.run(debug=True)
```

Step 1

Create Flask application object.

Step 2

Add `@app.route()` above a function.

Step 3

Return something that can be sent to the browser.

Practice rule

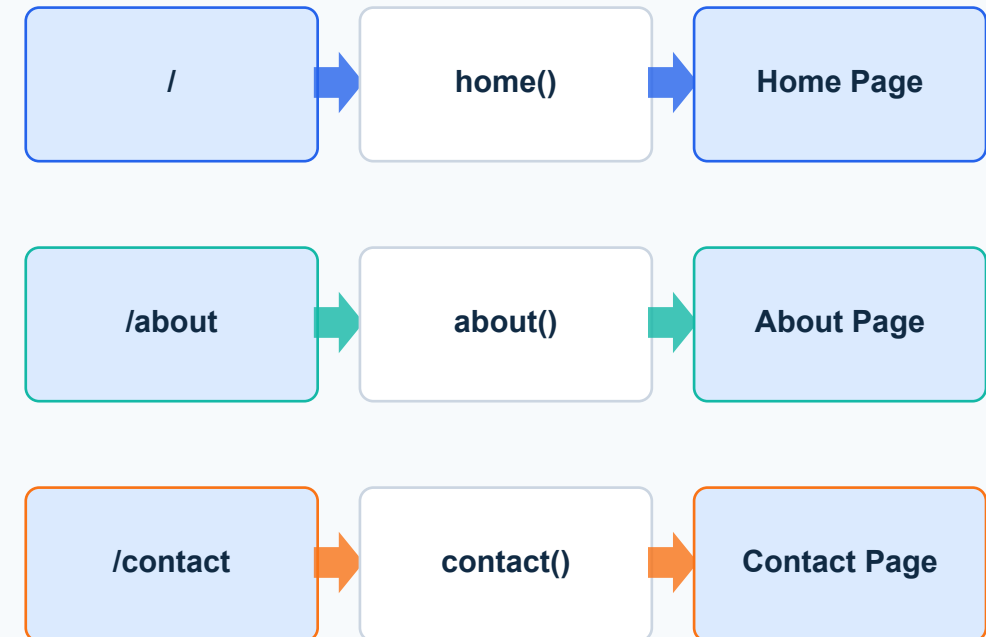
Change the URL and function name, then test in browser.

Static routing

Use static routes when the URL path is fixed.



```
1 @app.route('/')
2 def home():
3     return 'Home Page'
4
5 @app.route('/about')
6 def about():
7     return 'About Page'
8
9 @app.route('/contact')
10 def contact():
11     return 'Contact Page'
```



Good for fixed pages: home, about, contact, courses, placements, support.

Multiple routes for one function

One view function can respond to more than one URL.



```
1 @app.route('/')
2 @app.route('/home')
3 def home():
4     return 'Welcome Home'
```

What happens?

Both / and /home execute the same Python function.



Dynamic routing

Use dynamic routes when part of the URL changes.



Static URL

/student
Same page for everyone.

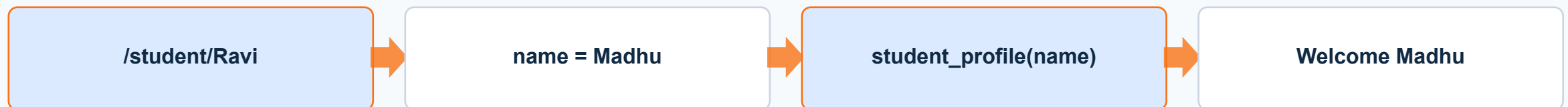
Dynamic URL

/student/101
/student/102
Different value captured.

Route variable

<name> captures part of the URL and sends it to the function.

```
1 @app.route('/student/<name>')  
2 def student_profile(name):  
3     return f'Welcome {name}'
```



URL parameters: variable names must match

The variable inside < > must be accepted by the function.



```
1 # Correct
2 @app.route('/course/<course_name>')
3 def course(course_name):
4     return f'Course: {course_name}'
```

```
1 # Wrong
2 @app.route('/course/<course_name>')
3 def course(name):
4     return f'Course: {name}'
```

Exam + coding trap

If Flask captures `course_name`, the function should receive `course_name`.

Error symptom

Students may see `TypeError: missing required positional argument or unexpected keyword argument`.

Debug habit

Compare route variable names with function parameters before blaming Flask.

URL converters overview

Converters validate and convert route values before the function runs.



Converter	Accepts	Example URL
string	Text without slash	/user/madhu
int	Integer numbers	/student/54
float	Decimal numbers	/price/99.5
path	Text including slashes	/files/a/b/c
uuid	UUID format	/item/550e8400-e29b-41d4-a716-446655440000

Key rule

If the value does not match the converter, Flask will not execute the function for that URL.

int converter

Use int when the URL value must be a whole number.



```
1 @app.route('/student/<int:student_id>')
2 def student(student_id):
3     return f'Student ID: {student_id}'
```

Works

/student/101

Does not match

/student/ravi

Why useful?

Prevents invalid text values from reaching business logic.

Real use

Student ID, product ID, order ID, batch number.

Practice

Create /marks/<int:score> and return the score.

float converter

Use float when the URL value can contain decimals.



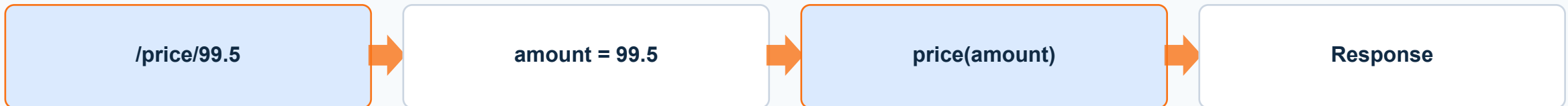
```
1 @app.route('/price/<float:amount>')
2 def price(amount):
3     return f'Price is {amount}'
```

Works

/price/49.99

Does not match

/price/free



Trainer warning

path converter

Use path when the captured value may contain slashes.



```
1 @app.route('/files/<path:file_path>')
2 def show_file(file_path):
3     return f'File path: {file_path}'
```

Example

/files/python/flask/day3.txt

Captured value

python/flask/day3.txt

string

Captures text until slash.

path

Captures text including slash.

Use case

File paths, nested categories, documentation pages.

uuid converter

Use uuid when IDs must follow UUID format.



```
1 @app.route('/ticket/<uuid:ticket_id>')
2 def ticket(ticket_id):
3     return f'Ticket: {ticket_id}'
```

Works

/ticket/550e8400-e29b-41d4-a716-446655440000

Does not match

/ticket/12345

Why UUID?

Hard to guess. Useful for public links and tokens.

Important

Flask passes it as UUID object, not plain random text.

Later use

Invoices, tickets, shareable links, password reset flows.

Trailing slash recap

Day 1 introduced this; Day 3 students must now test it by coding.



```
1 @app.route('/about')
2 def about():
3     return 'About'
```

Exact path

/about works. /about/ does not match this route.

```
1 @app.route('/about/')
2 def about():
3     return 'About'
```

Directory-like path

/about/ works. /about redirects to /about/.

Memory: slash means directory; Flask redirects to the canonical slash URL.

Routing conflict and order

Clear routes reduce confusion and make applications predictable.



```
1 @app.route('/user/<name>')
2 def user_by_name(name):
3     return f'Name: {name}'
4
5 @app.route('/user/settings')
6 def user_settings():
7     return 'Settings'
```

Potential confusion

/user/settings may look like name = settings if routes are not designed carefully.

Best practice

Keep fixed routes clear and meaningful. Use converters where possible.

Teaching line

A URL design is not only syntax. It is communication between developer, browser and user.

Live coding project: Student Routing App

Build this during class and ask students to predict each output before running.



```
1 from flask import Flask
2 app = Flask(__name__)
3
4 @app.route('/')
5 def home():
6     return 'Student Routing App'
7
8 @app.route('/student/<int:student_id>')
9 def student(student_id):
10     return f'Student ID: {student_id}'
11
12 @app.route('/course/<course_name>')
13 def course(course_name):
14     return f'Course: {course_name}'
15
16 @app.route('/files/<path:file_path>')
17 def files(file_path):
18     return f'Path: {file_path}'
19
20 app.run(debug=True)
```

Test 1

/student/54 should work.

Test 2

/student/madhu should not match int route.

Test 3

/course/flask should return course name.

Test 4

/files/a/b/c should capture a/b/c.

Classroom practice plan

Keep practice simple, visible and measurable.



Round 1 - Static

Create /, /about, /contact and /courses routes.

Round 2 - Dynamic

Create /student/<name> and /batch/<batch_code>.

Round 3 - Converters

Create /marks/<int:score> and /price/<float:amount>.

Round 4 - Path

Create /docs/<path:page_path>.

Round 5 - Debug

Break one variable name and fix the error.

Round 6 - Explain

Each student explains one route in plain English.

Submission: app.py screenshot + browser output screenshot + GitHub commit.

Common mistakes to catch quickly

Use this as your debugging checklist during labs.



1. Forgetting to start the path with / inside `@app.route()`.
2. Using route variable `<name>` but function parameter `student_name`.
3. Expecting `/about/` to work when only `/about` is defined.
4. Passing text into an `<int:id>` route.
5. Creating duplicate function names for different routes.
6. Not saving the file before refreshing the browser.

Trainer instruction

When students get an error, do not fix it immediately. Ask them to read the route and URL aloud.

Debugging habit

Check URL -> route rule -> variable name -> converter -> function parameter.

Outcome

Students should become comfortable with 404 and `TypeError` as learning signals.

Exit assessment and assignment

End with code, not theory.



In-class assessment

Build 6 routes:

1. /
2. /about
3. /student/<int:id>
4. /course/<name>
5. /price/<float:amount>
6. /files/<path:file_path>

Acceptance criteria

Application runs in debug mode.
All valid URLs work.
Invalid converter values do not match.
Student can explain one dynamic route.

Homework

Create a Course Routing App with course name, trainer name and duration routes.

GitHub evidence

Commit message: Completed Flask Day 3 Routing Practice

Tomorrow: URL building, redirect and HTTP methods.

Day 3 quick recap

Students should be able to answer these before leaving.



1. What is routing in Flask?
2. What is the difference between URL path and endpoint?
3. How do we create a static route?
4. How do we capture values from a URL?
5. When should we use int, float, path and uuid converters?
6. What is the trailing slash behavior in Flask?

```
1 @app.route('/student/<int:id>')
2 def student(id):
3     return f'Student: {id}'
```

Final memory line

Route = URL rule + Python function + response.

Next class

url_for(), redirect(), GET and POST methods.